

Politechnika Świętokrzyska w Kielcach
Wydział Elektrotechniki, Automatyki i Informatyki



Politechnika Świętokrzyska
Kielce University of Technology

Technologie obiektowe

Projekt

Generacja schematu dla dokumentów json

Skład zespołu:

Wiktor Syska

Kacper Michalak

Kierunek/specjalność: Informatyka/Systemy Informacyjne

Studia: stacjonarne II Stopnia

Numer grupy: 1ID21B

Spis treści

1.	Część teoretyczna	3
1.1.	Format Json	3
1.2.	Json Schema	3
2.	Implementacja.....	4
2.1.	Architektura projektu.....	4
2.2.	Technologie	4
2.3.	Moduły systemu	5
2.4.	Diagramy UML	7
2.5.	Parser.....	9
2.6.	Generator schematu.....	12
3.	Instrukcja użytkowania	16
4.	Testy	21

1. Część teoretyczna

1.1. Format Json

JSON (JavaScript Object Notation) to lekki, tekstowy format wymiany danych, zdefiniowany w standardzie RFC 8259. Pierwotnie wywodzi się z języka JavaScript, jednak ze względu na swoją prostotę i czytelność stał się niezależnym standardem wykorzystywanym w większości współczesnych systemów – od interfejsów API, przez pliki konfiguracyjne, aż po bazy dokumentowe takie jak MongoDB.

Struktura formatu opiera się na dwóch podstawowych konstrukcjach: obiektach oraz tablicach. Typy wartości obsługiwane w JSON to:

- string - ciąg znaków ujęty w cudzysłowy,
- number - liczba całkowita lub zmiennoprzecinkowa,
- boolean - wartość logiczna,
- null - wartość pusta,
- object - obiekt zawierający pary klucz–wartość,
- array - uporządkowana lista wartości.

Dzięki minimalnej gramatyce format ten jest łatwy do parsowania oraz niezależny od języka programowania, co czyni go naturalnym wyborem dla aplikacji wymagających serializacji danych.

1.2. Json Schema

JSON Schema to standard opisu struktury dokumentów JSON, utrzymywany przez grupę JSON Schema Organization. Pozwala zdefiniować jakie pola są oczekiwane w dokumencie, jakiego są typu, jakie wartości mogą przyjmować oraz jakie formaty powinny spełniać. W projekcie wykorzystana została specyfikacja draft 2020-12.

Sam schemat również jest dokumentem JSON. Minimalny schemat opisujący obiekt z polami wygląda następująco:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "id": { "type": "integer" },
    "name": { "type": "string" }
  }
}
```

Schemat pełni dwie główne role. Po pierwsze, dokumentuje oczekiwaną strukturę danych, ułatwiając współpracę między systemami. Po drugie, umożliwia walidację, czyli sprawdzenie czy dany dokument JSON spełnia zdefiniowane wymagania. Celem niniejszego projektu jest zarówno automatyczne generowanie takich schematów na podstawie przykładowych dokumentów, jak i ich późniejsze wykorzystanie do walidacji.

2. Implementacja

2.1. Architektura projektu

Aplikacja została zaprojektowana w sposób modułowy, zgodnie z zasadą rozdzielania odpowiedzialności. Każdy z głównych komponentów systemu odpowiada za jeden, ściśle zdefiniowany etap przetwarzania dokumentu JSON, od jego wczytania z pliku, przez parsowanie i budowę struktury obiektowej, aż po wygenerowanie schematu i jego ewentualną walidację. Takie podejście ułatwia testowanie poszczególnych elementów w izolacji oraz pozwala na wymianę lub rozbudowę pojedynczego modułu bez konieczności modyfikowania pozostałej części systemu.

Architekturę systemu można podzielić na następujące warstwy:

- Warstwa prezentacji – zrealizowana w JavaFX odpowiada za interakcję z użytkownikiem
- Warstwa logiki biznesowej – składa się z modułów odpowiedzialnych za przetwarzanie danych, generowanie schematów oraz walidację
- Warstwa modelu danych – reprezentuje ona struktury dokumentów JSON

Istotnym założeniem projektowym jest minimalizacja zależności od bibliotek zewnętrznych. Kluczowe komponenty odpowiedzialne za parsowanie dokumentu JSON oraz jego serializację zostały zaimplementowane samodzielnie, bez wykorzystania popularnych bibliotek typu Jackson czy Gson. Pozwoliło to na pełną kontrolę nad zachowaniem parsera oraz na dostosowanie modelu danych do specyficznych potrzeb generatora schematu.

Warstwa graficzna oparta na JavaFX została oddzielona od logiki biznesowej. Klasa kontrolera pełni rolę pośrednika - reaguje na zdarzenia interfejsu użytkownika i deleguje wykonanie konkretnych operacji do dedykowanych klas usługowych: JsonFileLoader, JsonParser, SchemaGenerator oraz JsonSchemaValidator. Dzięki temu logika aplikacji jest niezależna od konkretnej technologii UI i może być testowana bez uruchamiania okna aplikacji.

2.2. Technologie

Zestawienie technologii wykorzystanych w projekcie przedstawia poniższa tabela:

Technologia	Rola w projekcie
Java 21	główny język implementacji
JavaFX 21	interfejs graficzny użytkownika
FXML	deklaratywny opis układu okna
Maven	zarządzanie zależnościami i budowanie projektu
JUnit 5	framework testów jednostkowych

2.3. Moduły systemu

Kod źródłowy projektu został zorganizowany w następujące pakiety w przestrzeni `com.example.jsonschemagenerator`:

loader - odpowiada za wczytywanie plików JSON z dysku. Klasa `JsonFileLoader` weryfikuje poprawność pliku oraz udostępnia metody do ładowania pojedynczych plików i ich wsadowego przetwarzania. Dedykowany wyjątek `JsonLoadException` pozwala na precyzyjną obsługę błędów wejścia/wyjścia.

json - zawiera własny model obiektowy reprezentujący struktury JSON. Abstrakcyjna klasa `JsonValue` stanowi wspólny typ bazowy dla sześciu konkretnych implementacji odpowiadających typom JSON: `JsonObject`, `JsonArray`, `JsonString`, `JsonNumber`, `JsonBoolean` oraz `JsonNull`. W tym pakiecie znajduje się również własna klasa `ObjectMapper` odpowiedzialna za serializację modelu z powrotem do tekstowej reprezentacji JSON, zarówno w trybie zwartym, jak i z formatowaniem.

parser - implementuje ręcznie napisany parser dokumentów JSON. Klasa `JsonParser` na podstawie tekstowego wejścia buduje drzewo obiektów typu `JsonValue`. Pakiet zawiera także własny wyjątek `JsonParserException` sygnalizujący błędy składniowe.

generator - zawiera klasę `SchemaGenerator` odpowiedzialną za przekształcanie drzewa `JsonValue` na dokument JSON Schema zgodny ze specyfikacją draft 2020-12. W podpakiecie `dateValidator` znajduje się klasa `DateValidator` wykrywająca czy wartość tekstowa odpowiada formatowi daty lub daty z czasem (ISO 8601) i pozwalająca wzbogacić schemat o pole `format`.

validator - udostępnia klasę `JsonSchemaValidator`, która na podstawie wygenerowanego schematu sprawdza zgodność dowolnego dokumentu JSON. Walidator zwraca listę znalezionych błędów wraz z ścieżką w dokumencie, pod którą wystąpiła niezgodność.

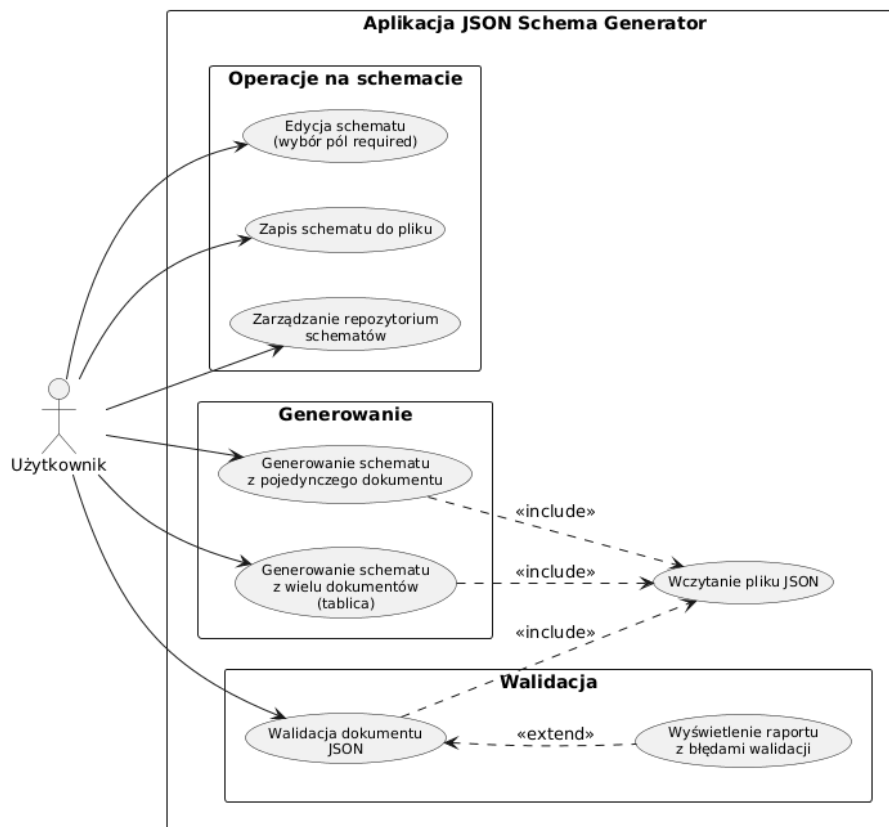
warstwa prezentacji - obejmuje klasy `HelloApplication`, `Launcher` oraz `HelloController`, a także plik widoku `hello-view.fxml`. Odpowiadają one za inicjalizację aplikacji JavaFX, prezentację interfejsu użytkownika oraz pośredniczenie pomiędzy zdarzeniami GUI a logiką biznesową.

database - odpowiada za trwałe przechowywanie wygenerowanych schematów oraz ich powiązanie z dokumentami JSON. Centralnym elementem pakietu jest klasa `SchemaRepository`, pełniąca rolę lokalnego repozytorium schematów. Przechowuje ona schematy zarówno w pamięci, w postaci mapy nazwa–schemat, jak i na dysku, gdzie każdy schemat zapisywany jest jako osobny plik `.json` w katalogu `.jsonschemagenerator/schemas` w folderze domowym użytkownika. Wykorzystanie stałej, bezwzględnej ścieżki opartej o katalog domowy gwarantuje spójne działanie zapisu i odczytu niezależnie od katalogu, z którego uruchomiono aplikację.

Repozytorium udostępnia operacje zapisu, odczytu, usuwania oraz wylistowania zapisanych schematów. Nazwy schematów podlegają walidacji – dozwolone są wyłącznie litery, cyfry oraz znaki podkreślenia i myślnika, co zabezpiecza przed błędnymi nazwami plików oraz

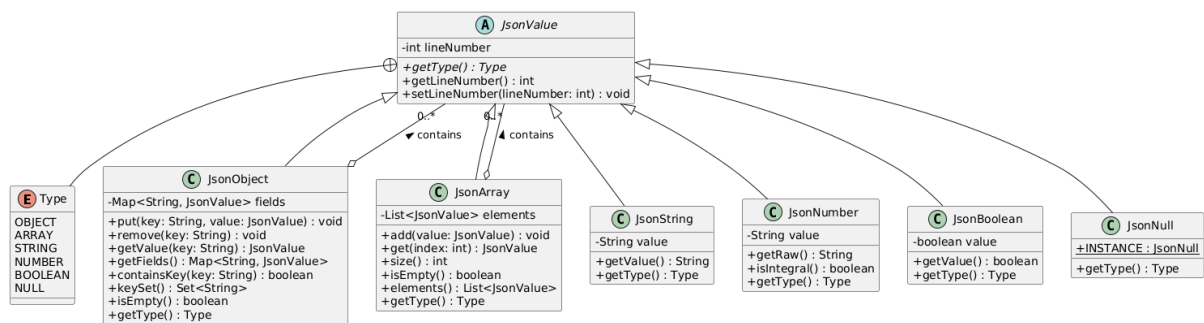
przed próbą wyjścia poza katalog repozytorium. Przy starcie aplikacji metoda loadAll wczytuje wszystkie schematy zapisane wcześniej na dysku.

2.4. Diagramy UML



Rysunek 2.1 Diagram przypadków użycia

Diagram przypadków użycia przedstawia funkcjonalności systemu. Grupa Operacje na schemacie obejmuje edycję wygenerowanego schematu, zapis schematu do pliku oraz zarządzanie lokalnym repozytorium schematów. Grupa Generowanie umożliwia utworzenie schematu JSON Schema na podstawie pojedynczego dokumentu lub tablicy wielu dokumentów. Grupa Walidacja pozwala na sprawdzenie zgodności dowolnego pliku JSON z aktualnym schematem, w przypadku wykrycia błędów system rozszerza ten przypadek o wyświetlenie znalezionych błędów. Wszystkie operacje generowania oraz walidacji wymagają wcześniejszego wczytania pliku JSON z dysku.



Rysunek 2.2 Diagram struktury danych

Diagram przedstawia strukturę modelu danych JSON zaimplementowanego w pakiecie `json`, opartego na wzorcu projektowym Kompozyt. Podstawą modelu danych jest abstrakcyjna klasa `JsonValue`. Dziedziczą z niej klasy reprezentujące poszczególne typy danych JSON. Klasy `JsonObject` oraz `JsonArray` pełnią rolę kompozytów, ponieważ mogą zawierać w sobie dowolne inne obiekty typu `JsonValue`. Taka architektura umożliwia rekurencyjne budowanie drzew o dowolnej głębokości zagnieżdżenia.

2.5. Parser

Parser został zaimplementowany samodzielnie w klasie `JsonParser`, bez korzystania z zewnętrznych bibliotek. Zastosowano w nim technikę rekurencyjnego zejścia, w której dla każdej reguły gramatyki JSON istnieje odpowiadająca jej metoda. Parser przechowuje wejściowy łańcuch znaków oraz aktualną pozycję analizy i przesuwa ją w miarę rozpoznawania kolejnych tokenów.

Główną metodą parsera jest `parseValue`, która na podstawie pierwszego niepustego znaku rozpoznaje typ kolejnej wartości i deleguje analizę do odpowiedniej metody specjalistycznej: `parseObject`, `parseArray`, `parseString`, `parseNumber`, `parseBool` lub `parseNull`. Metody te zwracają obiekty modelu `JsonValue`, dzięki czemu wynikiem parsowania jest pełne drzewo reprezentujące strukturę dokumentu wejściowego.

```
private JsonValue parseValue() throws JsonParserException, Json-
ParseException {
    skipWhiteSpace();

    int line = currentLine();
    if (position >= input.length()) {
        throw new JsonParseException("Json parsing error");
    }

    char c = input.charAt(position);
    JsonValue result;

    switch (c) {
        case '{':
            result = parseObject();
            break;
        case '"':
            result = parseString();
            break;
        case '[':
            result = parseArray();
            break;
        case 't', 'f':
            result = parseBool();
            break;
        case 'n':
            result = parseNull();
            break;
        default:
            if (c == '-' || Character.isDigit(c)) {
                result = parseNumber();
                break;
            }
            throw new JsonParseException("Nieoczekiwany
znak");
    }

    result.setLineNumber(line);
    return result;
}
```

Listing Implementacja.1 Kod parseValue

Mechanizm rekurencji najlepiej widoczny jest w metodzie `parseObject`, która po rozpoznaniu klucza wywołuje ponownie `parseValue` dla wartości pola.

```
private JsonObject parseObject() throws JsonParseException, Json-
ParserException {

    JsonObject object = new JsonObject();

    isMatching('{');

    skipWhiteSpace();

    if (lookup() == '}') {
        position++;
        return object;
    }

    do {
        skipWhiteSpace();
        String key = parseStringValue();
        skipWhiteSpace();
        isMatching(':');
        JsonValue value = parseValue();
        object.put(key, value);
        skipWhiteSpace();

    } while (lookup() == ',' && position++ >= 0);

    isMatching('}');
    return object;
}
```

Listing Implementacja.2 Kod parseObject

Liczby są rozpoznawane wraz z opcjonalnym znakiem minus, częścią ułamkową oraz wykładnikiem dziesiętnym; ich wartości są przechowywane w klasie JsonNumber jako łańcuch znaków, co pozwala uniknąć utraty precyzji i później rozróżnić wartości całkowite od zmiennoprzecinkowych.

```
private JsonNumber parseNumber() {
    int start = position;

    if (lookup() == '-')
        position++;

    while (position < input.length() && Character.isDigit(input.charAt(position))) {
        position++;
    }

    if (position < input.length() && input.charAt(position) == '.') {
        position++;

        while (position < input.length() && Character.isDigit(input.charAt(position))) {
            position++;
        }
    }

    if (position < input.length() && input.charAt(position) == 'e') {
        position++;
        if (lookup() == '+' || lookup() == '-') {
            position++;
        }

        while (position < input.length() && Character.isDigit(input.charAt(position))) {
            position++;
        }
    }

    return new JsonNumber(input.substring(start, position));
}
```

Listing Implementacja.3 Kod parseNumber

W przypadku napotkania nieprawidłowej składni, brakującego ogranicznika, niezamkniętego łańcucha znaków lub nieoczekiwanego symbolu parser zgłasza wyjątek JsonParserException zawierający informację o pozycji, na której wystąpił błąd.

2.6. Generator schematu

Generowaniem schematu zajmuje się klasa `SchemaGenerator`. Na wejściu otrzymuje ona drzewo `JsonValue` zwrócone przez parser, a na wyjściu produkuje obiekt `JsonObject` reprezentujący gotowy dokument JSON Schema zgodny ze specyfikacją draft 2020-12. Schemat zawiera właściwość `$schema` wskazującą wersję standardu oraz opcjonalną właściwość `title`, jeśli tytuł został podany.

Punktem początkowym jest metoda `generate()`, która tworzy główny obiekt schematu i przekazuje proces dalej rozpoznanie otrzymanego typu do metody `setType()`.

```
public JsonObject generate(JsonValue node, String title){
    if(node == null){
        throw new IllegalArgumentException("Węzeł JSON nie
może być null");
    }
    JsonObject schema = new JsonObject();
    schema.put("$schema", new JsonString(SCHEMA_VERSION));

    if(title != null && !title.isBlank()){
        schema.put("title", new JsonString(title));
    }

    setType(node, schema);
    return schema;
}
```

Listing Implementacja.4 Kod generate

Metoda `setType()` pełni rolę dyspozytora na podstawie typu węzła w drzewie `JsonValue` wybiera odpowiednią strategię generowania schematu. Dla typów prostych takich jak `boolean` czy `null` schemat ogranicza się do ustawienia pola `type`. Natomiast dla typów złożonych `object`, `array` oraz łańcuchów znaków wywoływane są dedykowane metody.

```
private void setType(JsonValue node, JsonObject schema){
    switch (node.getType()){
        case STRING:
            setStringDetectFormat((JsonString) node, schema);
            break;
        case NUMBER:
            setTypeNumber((JsonNumber) node, schema);
            break;
        case BOOLEAN:
            schema.put("type", new JsonString("boolean"));
            break;
        case NULL:
            schema.put("type", new JsonString("null"));
            break;
        case OBJECT:
            setTypeObject((JsonObject) node, schema);
            break;
        case ARRAY:
            setTypeArray((JsonArray) node, schema);
            break;
    }
}
```

Listing Implementacja.5 Kod setType

Generator dla JSON rekurencyjnie przetwarza każde pole, tworzy podobiekt schematu i ponownie wywołuje metodę `setType()`, pozwala to na obsłużenie dowolnego poziomu zagnieżdżenia.

```
private void setTypeObject(JsonObject node, JsonObject schema){
    schema.put("type", new JsonString("object"));
    JsonObject properties = new JsonObject();
    node.getFields().forEach((fieldName, fieldValue) ->{
        JsonObject fieldSchema = new JsonObject();
        setType(fieldValue, fieldSchema);
        properties.put(fieldName, fieldSchema);
    });
    schema.put("properties", properties);
}
```

Listing Implementacja.6 Kod setTypeObject

Obsługiwane jest również generowanie schematu na podstawie wielu dokumentów jednocześnie. Metoda `generateFromMultiple()` analizuje tabelę JSON, na podstawie zebranych informacji o typach wszystkich pól automatycznie wyznacza listę pól które są wymagane, pole jest uważane za wymagane jeśli występuje w każdym dokumencie i nigdy nie przyjmuje wartości `null`.

2.7. Walidator

Walidacją dokumentów JSON względem wygenerowanego schematu zajmuje się klasa `JsonSchemaValidator`. Jej zadaniem jest rekurencyjne porównanie struktury i typów danych w dokumencie wejściowym z oczekiwaniami w schemacie. Wynikiem walidacji jest lista komunikatów o błędach, z których każdy zawiera ścieżkę w dokumencie (np. `$.address.city`) oraz numer linii, w której wykryto błąd.

Metoda `validate()` jest głównym wejściem dla walidatora, jej zadanie jest inicjalizacja listy błędów i rozpoczęcie rekurencyjnego przechodzenia przez drzewo od korzenia.

```
public List<String> validate(JsonObject jsonSchema, JsonValue
data){
    List<String> errors = new ArrayList<>();
    validateNode(jsonSchema, data, "$", errors);
    return errors;
}
```

Listing Implementacja.7 Kod validate

Główna logika walidacji znajduje się w metodzie `validateNode()`. W pierwszej kolejności sprawdzany jest typ węzła jeśli schemat definiuje oczekiwany typ za pomocą pola `type`, walidator porównuje go z faktycznym typem wartości w dokumencie. W przypadku niezgodności dodawany jest komunikat o błędzie wraz ze ścieżką i numerem linii, a dalsze sprawdzanie danego węzła jest przerywane.

```
if(jsonSchema.containsKey("type")){
    String expectedType = (JsonString) jsonSchema.get-
value("type").getValue();

    if(!matchType(expectedType, data)){
        errors.add("Błąd na [" + path + "] linia " +
data.getLineNumber() + " Oczekiwano typu: " + expectedType);
        return;
    }
}
```

Listing Implementacja.8 Fragment I kodu metody validateNode

Jeśli walidowany węzeł jest obiektem, walidator sprawdza dwa aspekty. Po pierwsze, czy wszystkie pola oznaczone w schemacie jako wymagane istnieją w dokumencie. Po drugie, czy każde pole obecne w dokumencie jest zdefiniowane w schemacie. Jeśli nie zgłaszane jest nieoczekiwane pole. Dla każdego zdefiniowanego pola walidator wywołuje się rekurencyjnie.

```
if(jsonSchema.containsKey("required")){
    JSONArray required = (JSONArray) jsonSchema.get-
value("required");

    for(int i = 0; i < required.size(); i++){
        JsonString requiredField = (JsonString) required.get(i);
        String expectedKey = requiredField.getValue();

        if(!dataObject.containsKey(expectedKey)){
            errors.add("Błąd na [" + path + "] linia " +
data.getLineNumber() +
" Oczekiwano typu: " +
expectedKey + " jest on wymagany");
        }
    }
}
```

Listing Implementacja.9 Fragment II kodu metody validateNode

Walidator obsługuje również tablice JSON. Po wykryciu że bieżący węzeł jest tablicą sprawdza obecność pola `items` w schemacie definiuje ono oczekiwaną strukturę poszczególnych elementów. Generator schematu w przypadku tablic jednorodnych zapisuje w polu `items` pełny schemat elementu, natomiast dla tablic zawierających mieszane typy ustawia wartość `true`, co oznacza akceptację dowolnego typu. Walidator rozróżnia oba przypadki dla schematu obiektowego iteruje po wszystkich elementach tablicy i waliduje każdy z nich niezależnie, natomiast dla wartości `true` pomija dalsze sprawdzanie.

```
if(data.getType() == JsonValue.Type.ARRAY){
    JsonArray dataArray = (JsonArray) data;

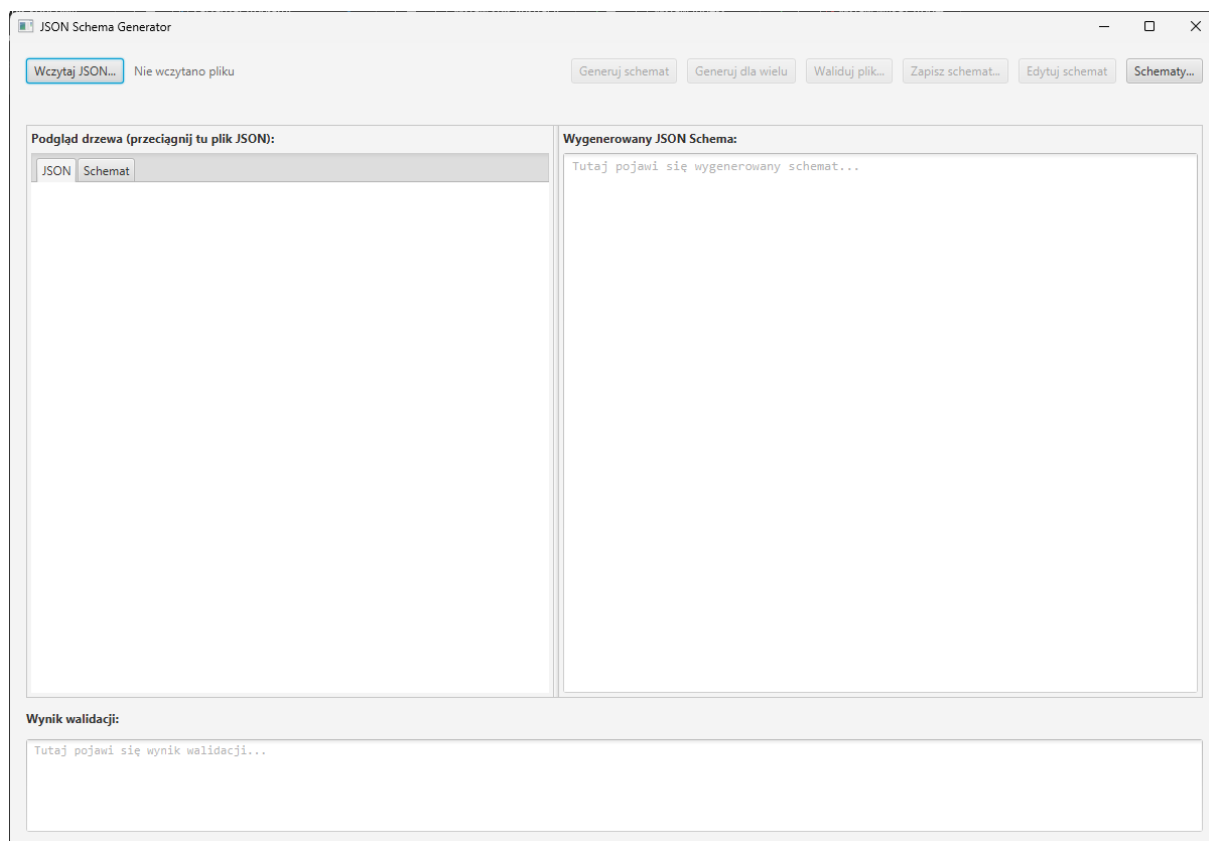
    if(jsonSchema.containsKey("items")){
        JsonValue itemsSchema = jsonSchema.get-
value("items");

        if(itemsSchema.getType() == JsonValue.Type.BOOL-
EAN)
            return;

        JsonObject itemSchemaObject = (JsonObject)
itemsSchema;
        for(int i = 0; i < dataArray.size(); i++){
            String currentPath = path + "[" + i + "]";
            validateNode(itemSchemaObject,
dataArray.get(i), currentPath, errors);
        }
    }
}
```

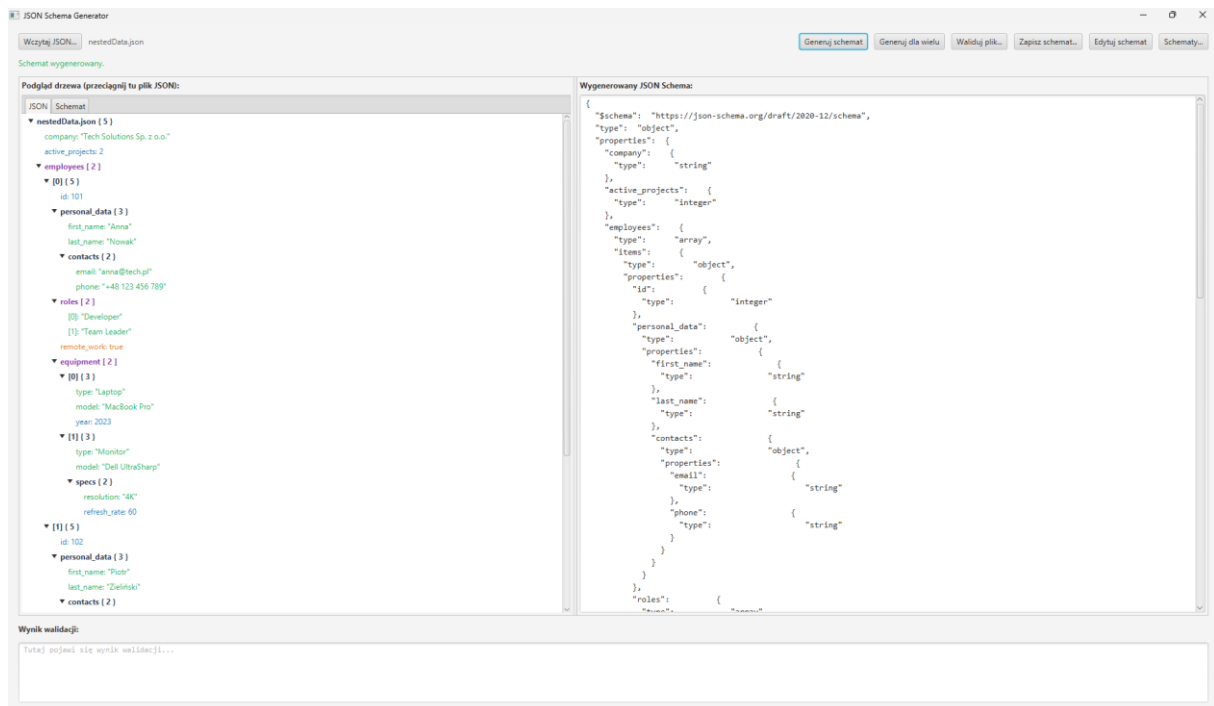
3. Instrukcja użytkowania

Po uruchomieniu aplikacji wyświetlane jest okno główne podzielone na trzy obszary. W górnej części znajduje się pasek przycisków oraz nazwa aktualnie wczytanego pliku. Część środkowa podzielona jest na dwie kolumny: po lewej stronie umieszczono widok drzewa z zakładkami „JSON” oraz „Schemat”, a po prawej pole tekstowe prezentujące wygenerowany schemat. W dolnej części okna znajduje się obszar prezentujący wynik walidacji. Pod paskiem przycisków wyświetlany jest również komunikat statusu informujący o wyniku ostatnio wykonanej operacji.



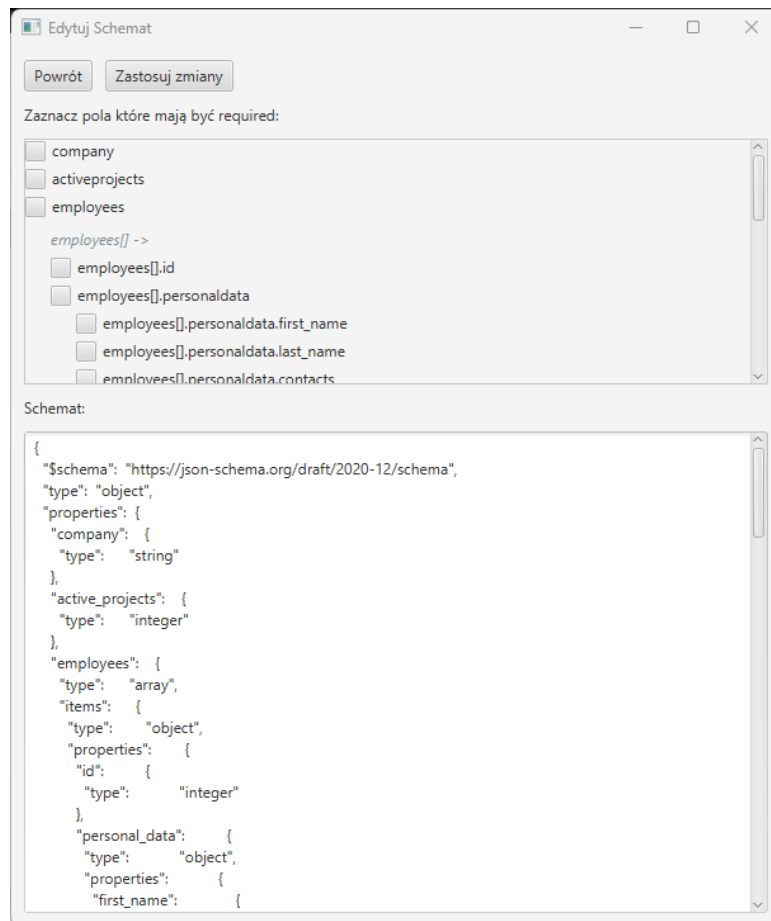
Rysunek 3.1 Okno główne

Wczytywanie pliku JSON. Pracę z aplikacją rozpoczyna się od wczytania dokumentu JSON. Można to zrobić na dwa sposoby: za pomocą przycisku „Wczytaj JSON...”, który otwiera systemowe okno wyboru pliku, albo przez przeciągnięcie pliku z menedżera plików i upuszczenie go na obszarze drzewa. Aplikacja zapamiętuje ostatnio używany katalog, dzięki czemu kolejne okna wyboru pliku otwierają się w tej samej lokalizacji. Po poprawnym wczytaniu struktura dokumentu zostaje wyświetlona w zakładce „JSON” w postaci kolorowanego drzewa, w którym poszczególne typy danych odróżnione są kolorem.



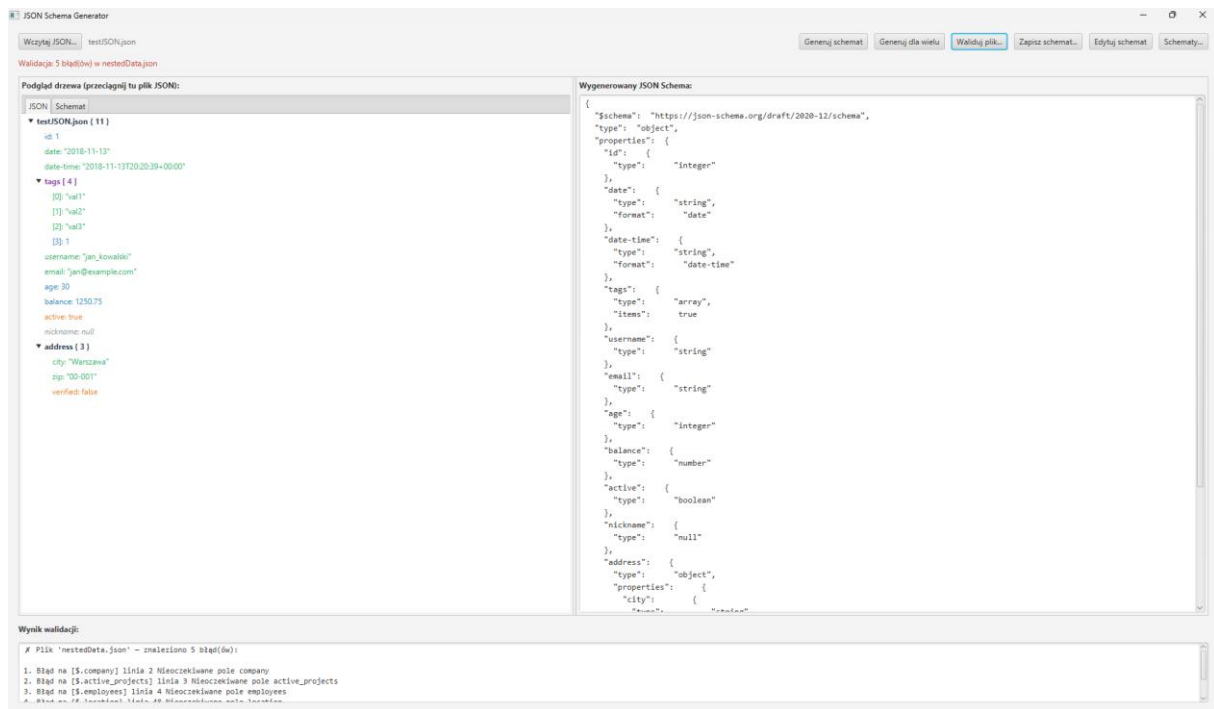
Rysunek 3.2 Okno po załadowaniu JSON i wygenerowaniu schematu

Generowanie schematu. Po wczytaniu pliku dostępne stają się przyciski generowania schematu. Przycisk „Generuj schemat” tworzy schemat na podstawie pojedynczego dokumentu. Przycisk „Generuj dla wielu” przeznaczony jest dla plików zawierających tablicę dokumentów - na ich podstawie powstaje wspólny schemat, w którym dodatkowo wyznaczana jest lista pól wymaganych. Wygenerowany schemat zostaje wyświetlony zarówno w postaci sformatowanego tekstu w prawej kolumnie okna, jak i w postaci drzewa w zakładce „Schemat”.



Rysunek 3.3 Okno edycji

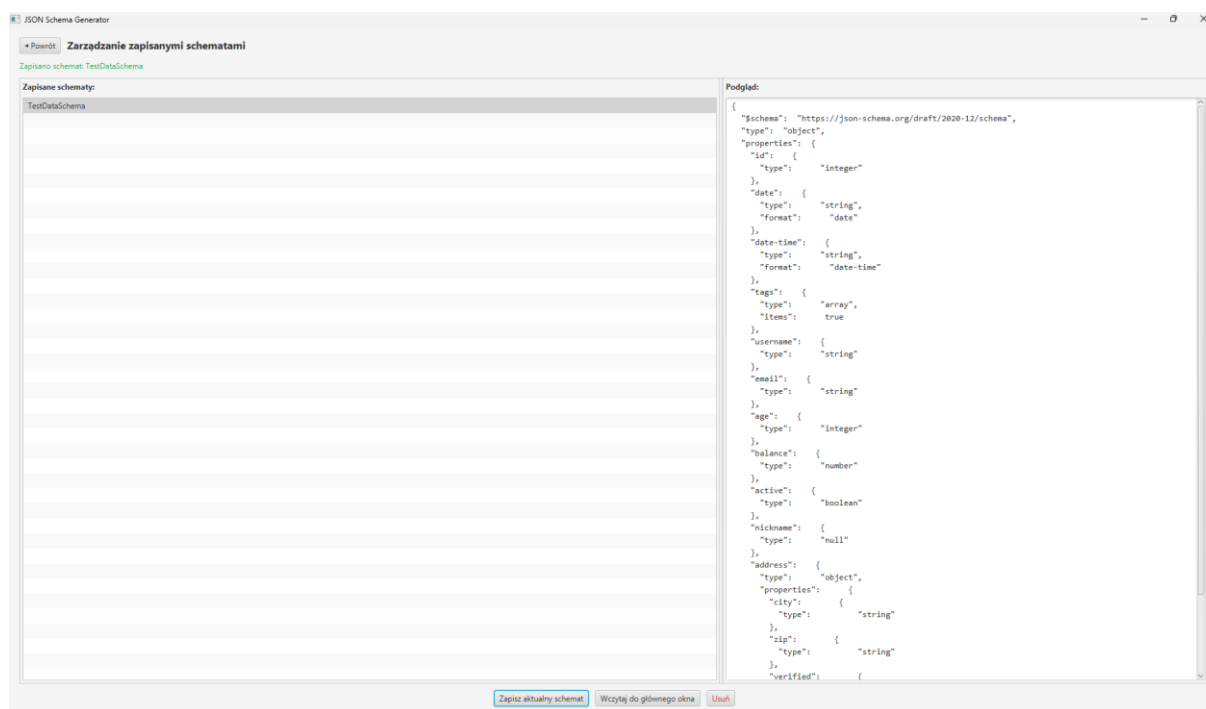
Edycja schematu. Przycisk „Edytuj schemat” otwiera dodatkowe okno pozwalające oznaczyć, które pola mają być wymagane. Pola prezentowane są w postaci listy pól wyboru odwzorowującej strukturę schematu, z uwzględnieniem zagnieżdżonych obiektów oraz elementów tablic. Po zatwierdzeniu zmian przyciskiem „Zastosuj zmiany” schemat zostaje zaktualizowany o właściwość required, a zmodyfikowana wersja jest widoczna w oknie głównym.



Rysunek 3.4 Okno po walidacji

Walidacja dokumentu. Gdy schemat został wygenerowany lub wczytany, dostępna staje się funkcja walidacji. Przycisk „Waliduj plik...” pozwala wskazać dowolny dokument JSON, który zostanie sprawdzony pod kątem zgodności z aktualnym schematem. Wynik prezentowany jest w dolnym obszarze okna: w przypadku zgodności wyświetlany jest komunikat potwierdzający, a w przypadku wykrycia niezgodności - ponumerowana lista błędów wraz ze ścieżką w dokumencie oraz numerem linii, w której wystąpił problem.

Zapis schematu do pliku. Wygenerowany schemat można zapisać na dysku przyciskiem „Zapisz schemat...”. Aplikacja proponuje domyślną nazwę pliku powiązaną z nazwą wczytanego dokumentu, a schemat zapisywany jest w czytelnej, sformatowanej postaci.



Rysunek 3.5 Okno zarządzania schematami

Zarządzanie repozytorium schematów. Przycisk „Schematy...” otwiera ekran zarządzania lokalnym repozytorium. Po lewej stronie znajduje się lista zapisanych schematów, a po prawej podgląd wybranego schematu. Z tego poziomu można zapisać aktualny schemat pod wybraną nazwą, wczytać zapisany schemat do okna głównego w celu wykorzystania go do walidacji, a także usunąć niepotrzebny schemat. Schematy przechowywane są w stałej lokalizacji w katalogu domowym użytkownika, dzięki czemu pozostają dostępne między kolejnymi uruchomieniami programu.

4. Testy

Do testów jednostkowych wykorzystano framework JUnit 5. Testy pokrywają najważniejsze moduły aplikacji i są uruchamiane automatycznie przy budowie projektu przez Maven.

W ramach pakietu testów jednostkowych przygotowano następujące klasy testowe:

JsonParserTest — weryfikuje poprawność działania parsera dla wszystkich obsługiwanych typów danych JSON. Testy obejmują m.in. parsowanie obiektów prostych i zagnieżdżonych, tablic jednorodnych i mieszanych, liczb całkowitych, ujemnych i dziesiętnych, wartości logicznych oraz null. Sprawdzane są również sytuacje błędne — pusty łańcuch wejściowy, brakujący nawias zamykający oraz brak dwukropka po kluczu — w których oczekiwany jest wyjątek.

JsonFileLoaderTest — pokrywa logikę modułu ładowania plików. Wykorzystuje mechanizm `@TempDir` z biblioteki JUnit 5 do tworzenia tymczasowych plików testowych. Sprawdza obsługę poprawnych plików JSON, ładowanie wsadowe wielu plików jednocześnie oraz reakcję na sytuacje wyjątkowe: nieistniejący plik, plik o nieprawidłowym rozszerzeniu, plik pusty, oraz przekazanie wartości null.

ObjectMapperTest — weryfikuje serializację modelu obiektowego do tekstowej reprezentacji JSON. Testy obejmują podstawowe przypadki, takie jak pusty obiekt i pusta tablica, a także bardziej złożone scenariusze, np. tablicę obiektów o wielu polach.

Testy zostały uruchomione na zestawie przykładowych dokumentów JSON umieszczonych w katalogu `src/main/resources`, obejmujących zarówno proste struktury z polami podstawowych typów, jak i bardziej złożone dokumenty z głębokim zagnieżdżeniem obiektów i tablic. Pozwala to zweryfikować zachowanie całej ścieżki przetwarzania — od ładowania pliku, przez parsowanie, aż po generowanie i serializację schematu.

Spis listingów:

Listing Błąd: nie znaleziono źródła odwołania.1 Kod parseValue.....	9
Listing Błąd: nie znaleziono źródła odwołania.2 Kod parseObject.....	10
Listing Błąd: nie znaleziono źródła odwołania.3 Kod parseNumber.....	11
Listing Błąd: nie znaleziono źródła odwołania.4 Kod generate.....	12
Listing Błąd: nie znaleziono źródła odwołania.5 Kod setType.....	12
Listing Błąd: nie znaleziono źródła odwołania.6 Kod setTypeObject	13
Listing Błąd: nie znaleziono źródła odwołania.7 Kod validate	13
Listing Błąd: nie znaleziono źródła odwołania.8 Fragment I kodu metody validateNode.....	14
Listing Błąd: nie znaleziono źródła odwołania.9 Fragment II kodu metody validateNode ...	14

Spis rysunków:

Rysunek 2.1 Diagram przypadków użycia	7
Rysunek 2.2 Diagram struktury danych	8
Rysunek 3.1 Okno główne.....	16
Rysunek 3.2 Okno po załadowaniu JSON i wygenerowaniu schematu	17
Rysunek 3.3 Okno edycji	18
Rysunek 3.4 Okno po walidacji.....	19
Rysunek 3.5 Okno zarządzania schematami	20